

Adv. DevOps Case Study: Serverless Application with Monitoring

Name: Rohan Lalchandani

Class: D15A Roll no. 25

1. Introduction

Case Study Overview:

This case study focuses on the integration of AWS Lambda, S3, and Nagios monitoring to create a seamless and efficient cloud-based solution for managing and processing images. AWS Lambda is leveraged to automatically trigger serverless functions in response to events, such as image uploads, while Amazon S3 serves as the scalable storage solution for these images. Nagios is implemented for real-time monitoring and alerting, ensuring system reliability and performance.

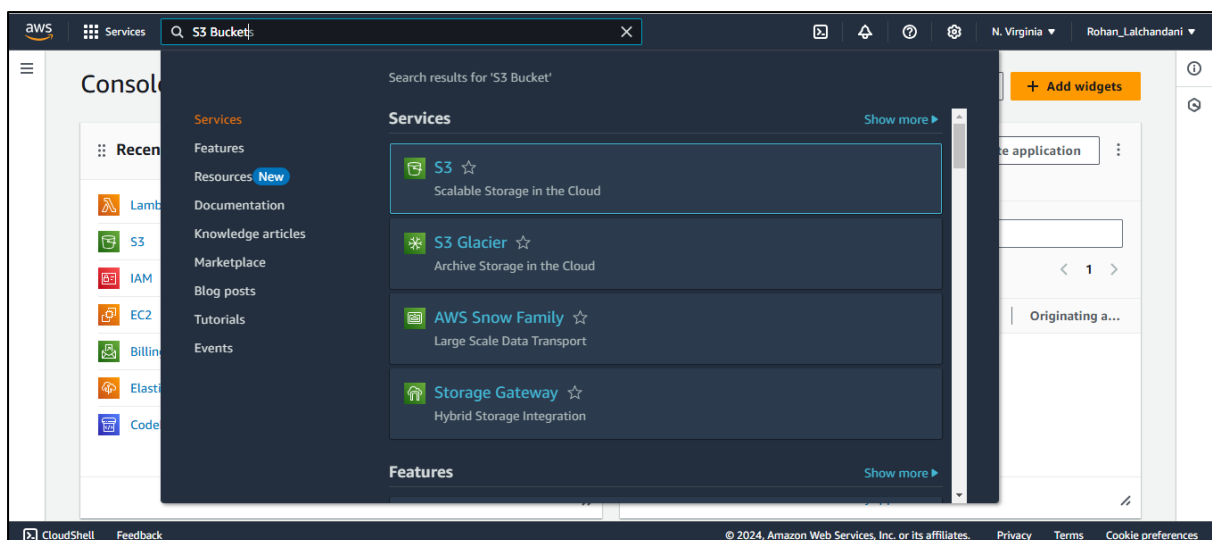
Key Feature and Application:

The unique feature of this solution is its ability to automate image processing workflows through serverless architecture, which significantly reduces operational overhead and improves scalability. This practical use case is particularly beneficial for applications that require quick processing of uploaded images, such as e-commerce platforms or media websites, where timely responses to user actions are critical for enhancing user experience.

2. Step-by-Step Explanation

Step 1: Create an S3 Bucket

1. Log into AWS Management Console.
2. Navigate to the S3 Service.
3. Click on Create Bucket.
4. Set a unique bucket name (e.g., image-upload-bucket).
5. Choose the region close to you.
6. Leave other settings as default unless your case study specifies otherwise.
7. Click Create Bucket.



aws

Services

Search

[Alt+S]

N. Virginia

Rohan_Lalchandani

Storage

Amazon S3

Store and retrieve any amount of data from anywhere

Amazon S3 is an object storage service that offers industry-leading scalability, data availability, security, and performance.

Create a bucket

Every object in S3 is stored in a bucket. To upload files and folders to S3, you'll need to create a bucket where the objects will be stored.

Create bucket

How it works

aws

Introduction to Amazon S3

Copy link

Pricing

With S3, there are no minimum fees. You only pay for what you use. Prices are based on the location of your S3 bucket.

Estimate your monthly bill using the [AWS Simple Monthly Calculator](#)

aws

Services

Search

[Alt+S]

N. Virginia

Rohan_Lalchandani

Create bucket

Info

Buckets are containers for data stored in S3.

General configuration

AWS Region

US East (N. Virginia) us-east-1

Bucket type

Info

☒ General purpose

Recommended for most use cases and access patterns. General purpose buckets are the original S3 bucket type. They allow a mix of storage classes that redundantly store objects across multiple Availability Zones.

☐ Directory

Recommended for low-latency use cases. These buckets use only the S3 Express One Zone storage class, which provides faster processing of data within a single Availability Zone.

Bucket name

Info

lambda-bucket-rohan

Bucket name must be unique within the global namespace and follow the bucket naming rules. [See rules for bucket naming](#)

Copy settings from existing bucket - optional

Only the bucket settings in the following configuration are copied.

Choose bucket

aws

Services

Search

[Alt+S]

N. Virginia

Rohan_Lalchandani

Successfully created bucket "lambda-bucket-rohan"

View details

To upload files and folders, or to configure additional bucket settings, choose **View details**.

Account snapshot - updated every 24 hours

All AWS Regions

View Storage Lens dashboard

Storage lens provides visibility into storage usage and activity trends. [Learn more](#)

General purpose buckets

Directory buckets

General purpose buckets (1)

Info

All AWS Regions

Refresh

Copy ARN

Empty

Delete

Create bucket

Find buckets by name

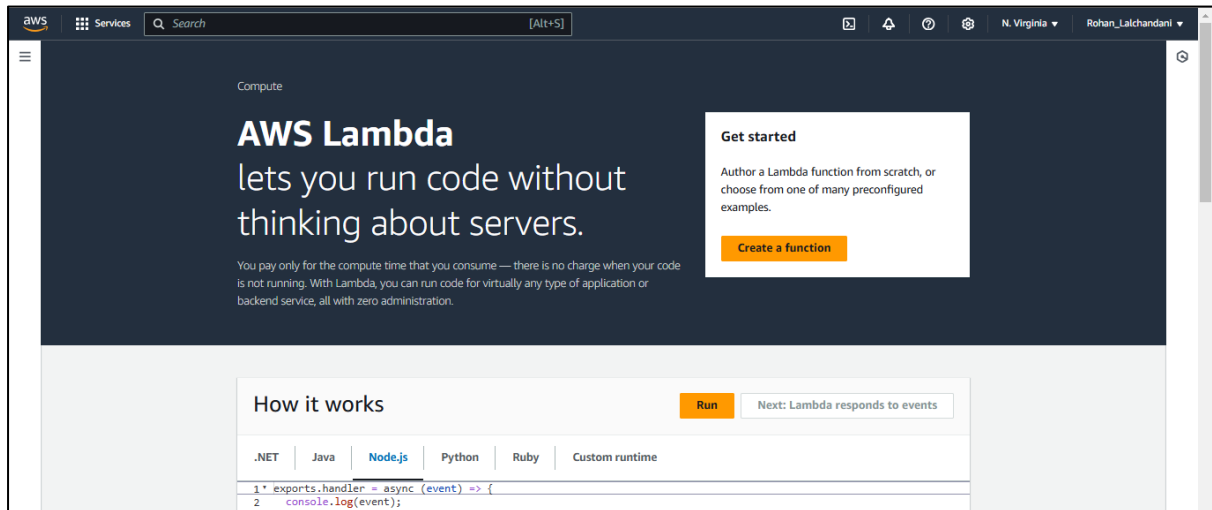
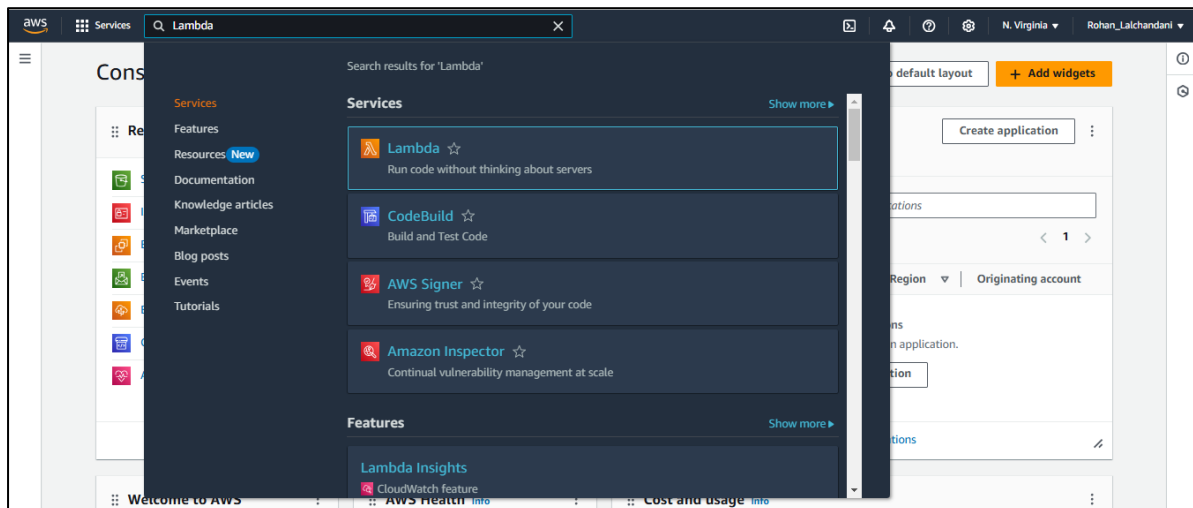
< 1 >

Settings

Name	AWS Region	IAM Access Analyzer	Creation date
☐ lambda-bucket-rohan	US East (N. Virginia) us-east-1	View analyzer for us-east-1	October 24, 2024, 21:25:45 (UTC+05:30)

Step 2: Create an AWS Lambda Function

1. **Navigate to AWS Lambda** in the AWS Console.
2. Click on **Create function**.
 - Choose **Author from scratch**.
 - Set function name (e.g., image-upload-logger).
 - Runtime: Select **Python 3.x**.
3. Click **Create function**.



Create function [Info](#)

Choose one of the following options to create your function.

- ☒ **Author from scratch**
Start with a simple Hello World example.
- ☐ **Use a blueprint**
Build a Lambda application from sample code and configuration presets for common use cases.
- ☐ **Container image**
Select a container image to deploy for your function.

Basic information

Function name
Enter a name that describes the purpose of your function.

 Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Runtime [Info](#)
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.
 [↻](#)

Architecture [Info](#)
Choose the instruction set architecture you want for your function code.
☒ x86_64
☐ arm64

Permissions [Info](#)
By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. You can customize this default role later when adding triggers.

▼ **Change default execution role**

Execution role
Choose a role that defines the permissions of your function. To create a custom role, go to the [IAM console](#).

- ☐ Create a new role with basic Lambda permissions
- ☐ Use an existing role
- ☒ Create a new role from AWS policy templates

[i](#) Role creation might take a few minutes. Please do not delete the role or edit the trust or permissions policies in this role.

Role name
Enter a name for your new role.

Under **Execution role**, choose **Create a new role with basic Lambda permissions** (or use an existing role if it has S3 permissions) search for “awss3bucketreadonlyaccess” policy and set it.

☒ **Create a new role from AWS policy templates**

[i](#) Role creation might take a few minutes. Please do not delete the role or edit the trust or permissions policies in this role.

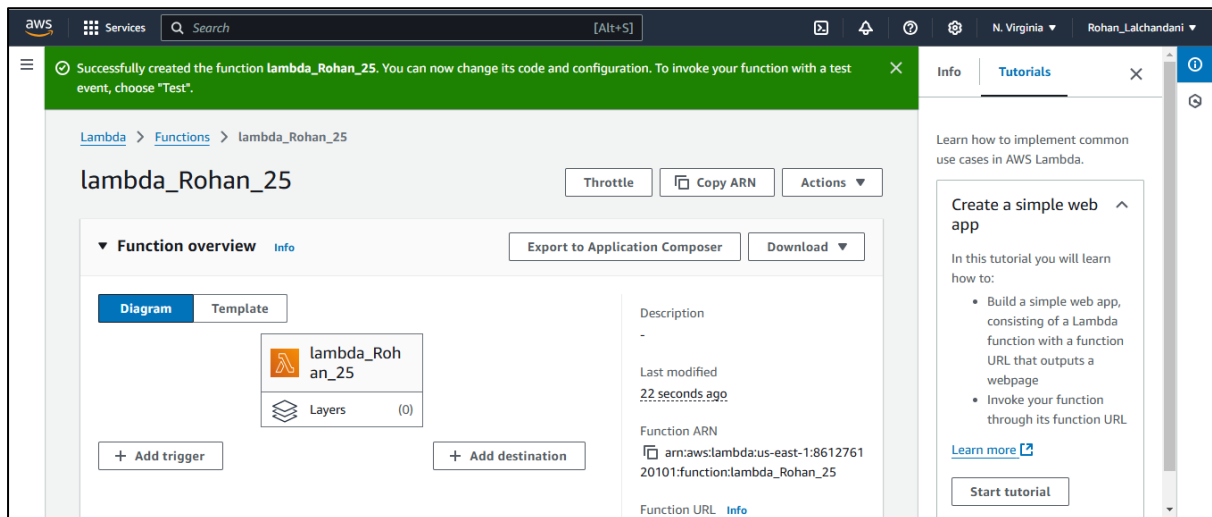
Role name
Enter a name for your new role.

 Use only letters, numbers, hyphens, or underscores with no spaces.

Policy templates - optional [Info](#)
Choose one or more policy templates.
 [↻](#)

► **Additional Configurations**
Use additional configurations to set up code signing, function URL, tags, and Amazon VPC access for your function.

[Cancel](#) [Create function](#)

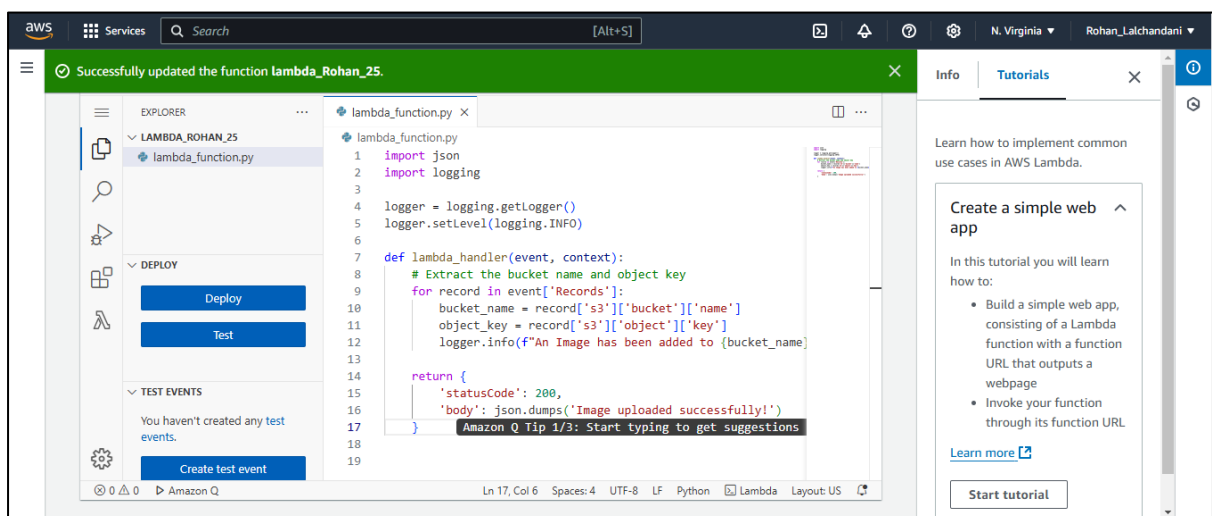


Step 3: Configure the Lambda Function Code

1. Scroll down to the function code editor and replace the default code with the following Python code:

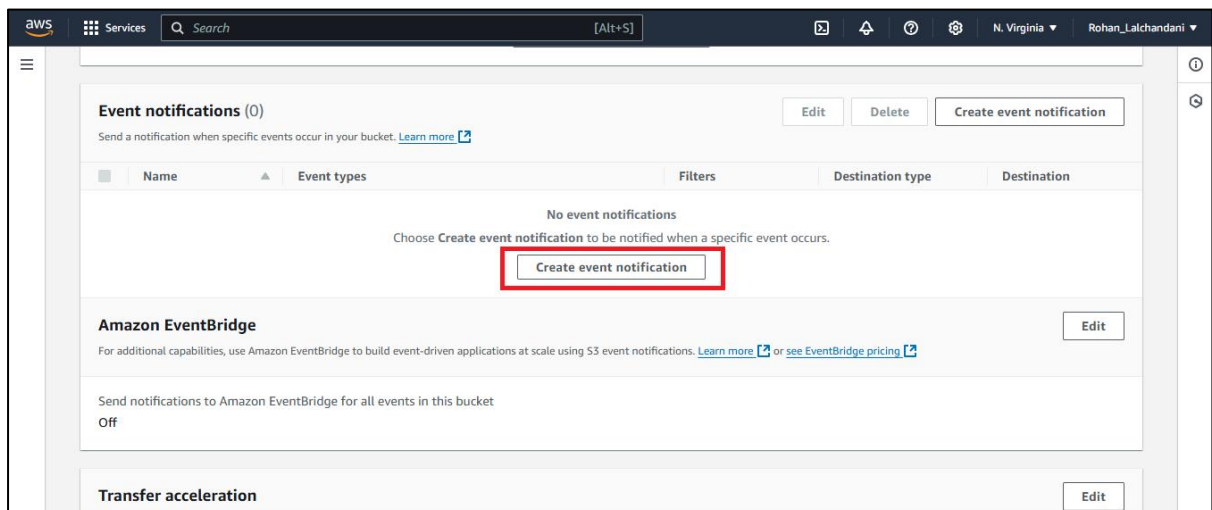
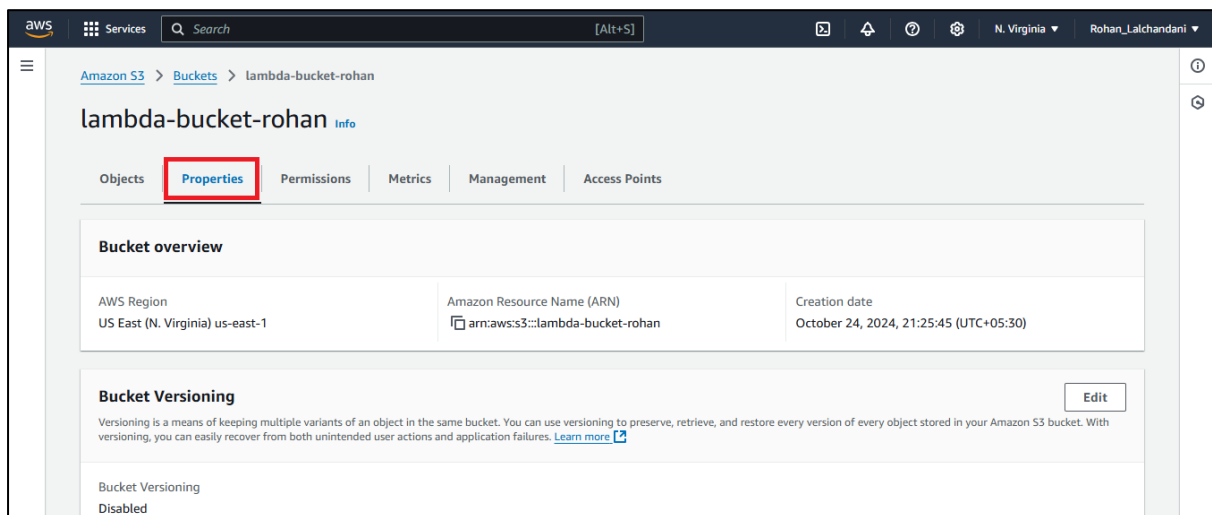
```
import json
import logging
logger = logging.getLogger()
logger.setLevel(logging.INFO)
def lambda_handler(event, context):
    # Extract the bucket name and object key
    for record in event['Records']:
        bucket_name = record['s3']['bucket']['name']
        object_key = record['s3']['object']['key']
        logger.info(f"An Image has been added to {bucket_name}, object key: {object_key}")

    return {
        'statusCode': 200,
        'body': json.dumps('Image uploaded successfully!')
    }
```



Step 4: Set up an S3 Event Notification

1. Navigate to the **S3 bucket** created in Step 1.
2. Go to the **Properties** tab, scroll to the bottom, and find **Event notifications**.
3. Click **Create Event Notification**.
 - Set a name for the event (e.g., image-upload-notification).
 - Select **All object create events**.
 - Under **Destination**, choose **Lambda Function**.
 - Choose the Lambda function created earlier (image-upload-logger).
4. Click **Save**.



aws

Services

Search

[Alt+S]

N. Virginia

Rohan_Lalchandani

Amazon S3

Buckets

lambda-bucket-rohan

Create event notification

Create event notification [Info](#)

To enable notifications, you must first add a notification configuration that identifies the events you want Amazon S3 to publish and the destinations where you want Amazon S3 to send the notifications.

General configuration

Event name

image-upload-notification

Event name can contain up to 255 characters.

Prefix - optional

Limit the notifications to objects with key starting with specified characters.

images/

Suffix - optional

Limit the notifications to objects with key ending with specified characters.

.jpg

aws

Services

Search

[Alt+S]

N. Virginia

Rohan_Lalchandani

Event types

Specify at least one event for which you want to receive notifications. For each group, you can choose an event type for all events, or you can choose one or more individual events.

Object creation

☒ All object create events

s3:ObjectCreated:*

☐ Put

s3:ObjectCreated:Put

☐ Post

s3:ObjectCreated:Post

☐ Copy

s3:ObjectCreated:Copy

☐ Multipart upload completed

s3:ObjectCreated:CompleteMultipartUpload

Object removal

☐ All object removal events

☐ Permanently deleted

aws

Services

Search

[Alt+S]

N. Virginia

Rohan_Lalchandani

Destination

Before Amazon S3 can publish messages to a destination, you must grant the Amazon S3 principal the necessary permissions to call the relevant API to publish messages to an SNS topic, an SQS queue, or a Lambda function. [Learn more](#)

Destination

Choose a destination to publish the event. [Learn more](#)

☒ Lambda function

Run a Lambda function script based on S3 events.

☐ SNS topic

Fanout messages to systems for parallel processing or directly to people.

☐ SQS queue

Send notifications to an SQS queue to be read by a server.

Specify Lambda function

☒ Choose from your Lambda functions

☐ Enter Lambda function ARN

Lambda function

aws

Services

Search

[Alt+S]

N. Virginia

Rohan_Lalchandani

Destination

Before Amazon S3 can publish messages to a destination, you must grant the Amazon S3 principal the necessary permissions to call the relevant API to publish messages to an SNS topic, an SQS queue, or a Lambda function. [Learn more](#)

Destination

Choose a destination to publish the event. [Learn more](#)

☒ Lambda function

Run a Lambda function script based on S3 events.

☐ SNS topic

Fanout messages to systems for parallel processing or directly to people.

☐ SQS queue

Send notifications to an SQS queue to be read by a server.

Specify Lambda function

☒ Choose from your Lambda functions

☐ Enter Lambda function ARN

Lambda function

lambda_Rohan_25

Cancel

Save changes

CloudShell

Feedback

© 2024, Amazon Web Services, Inc. or its affiliates. [Privacy](#) [Terms](#) [Cookie preferences](#)

The notification has been successfully created.

aws

Services

Search

[Alt+S]

N. Virginia

Rohan_Lalchandani

No data events to display.

Configure in CloudTrail

Event notifications (1)

Edit Delete Create event notification

☐

Name

Event types

Filters

Destination type

Destination

☐

image-upload-notification

All object create events

-

Lambda function

[lambda_Rohan_25](#)

Amazon EventBridge

Edit

For additional capabilities, use Amazon EventBridge to build event-driven applications at scale using S3 event notifications. [Learn more](#) or [see EventBridge pricing](#)

Send notifications to Amazon EventBridge for all events in this bucket

Off

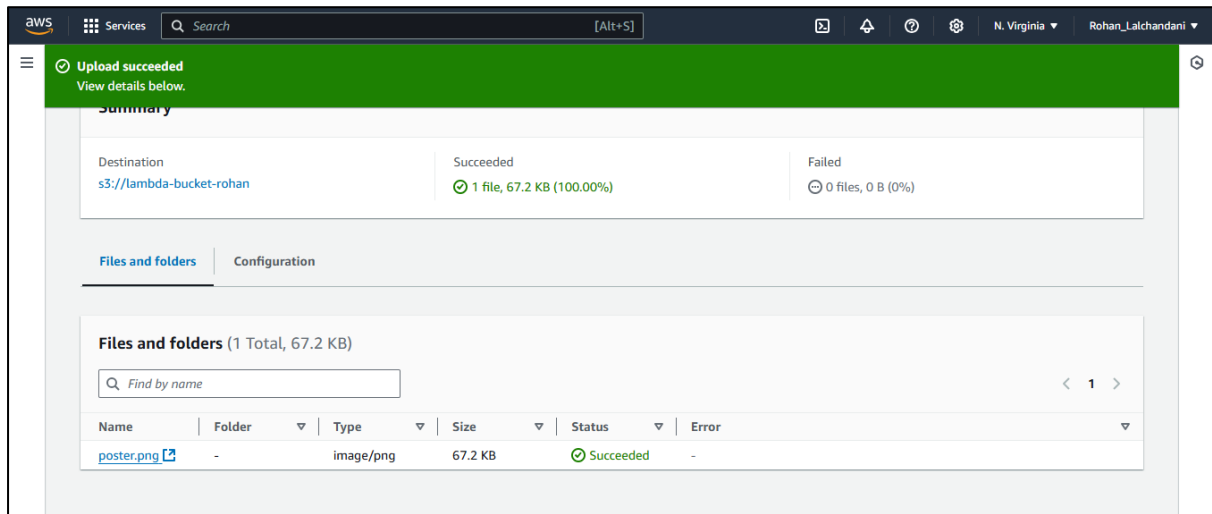
Transfer acceleration

Edit

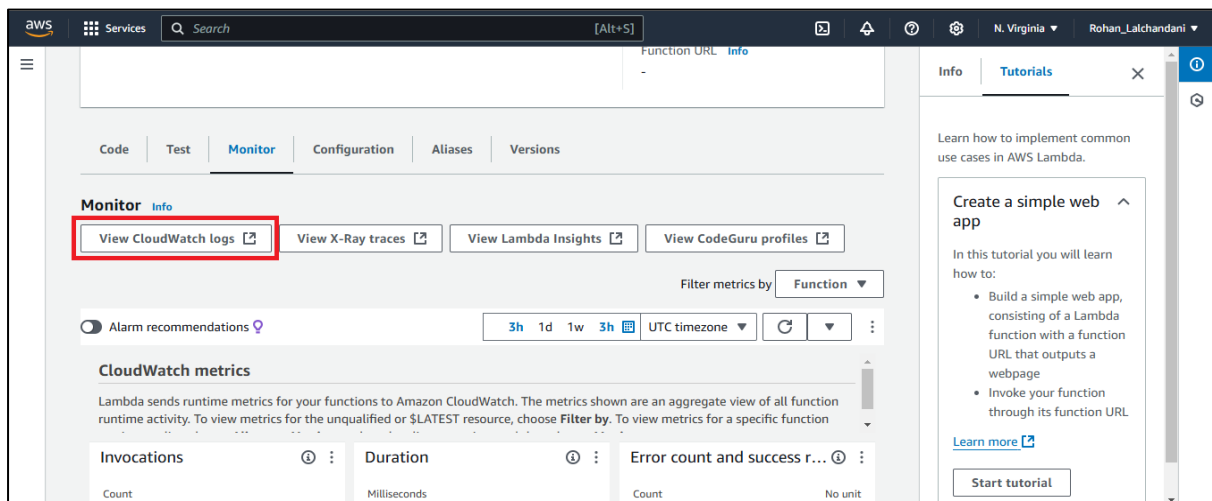
Use an accelerated endpoint for faster data transfers. [Learn more](#)

Testing the lambda function for logs:

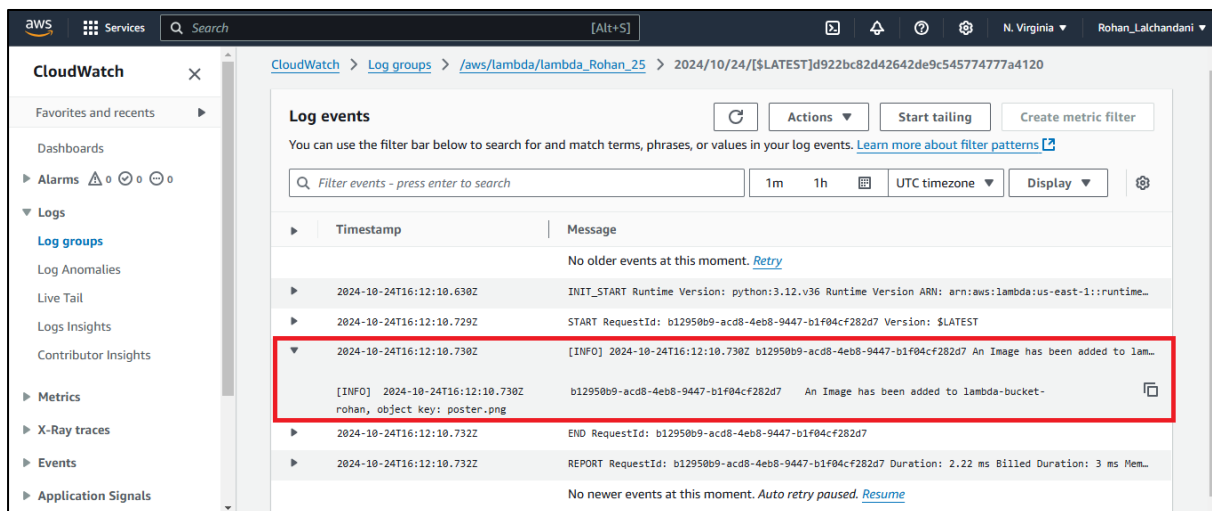
Upload an image to the S3 bucket



Go to the monitor section and click on CloudWatch logs



Under log events you should see the message that an image has ben uploaded.

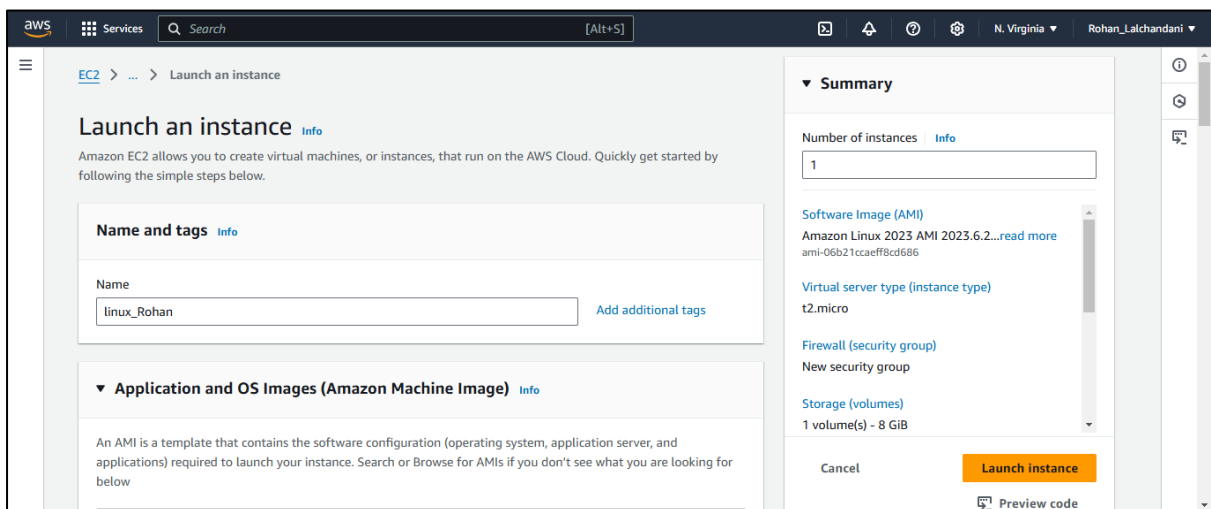
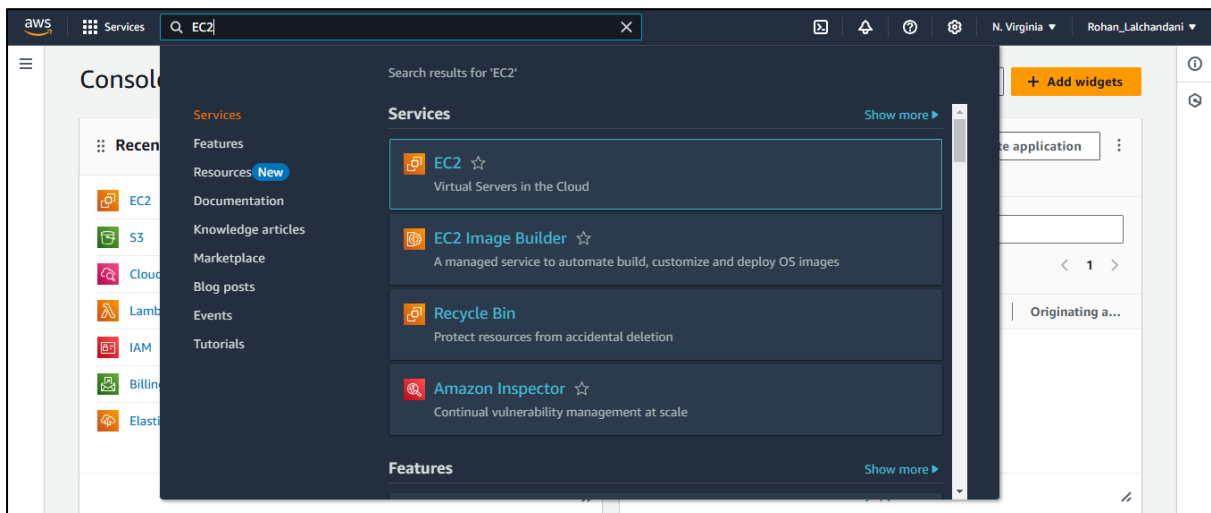


Steps to Install and Configure Nagios on AWS EC2 Instance and Monitor AWS Lambda Logs from S3 Bucket

Step 1: Set up an AWS EC2 Instance

1. Launch an EC2 Instance:

- Open the AWS Management Console and navigate to **EC2**.
- Click on **Launch Instance** and choose **Amazon Linux 2** or **Ubuntu 20.04 LTS** as the base image.
- Choose an instance type (e.g., t2.micro for free-tier).
- Configure the instance with default settings, or make changes as per your needs.
- Add necessary **security group** rules:
 - SSH (port 22) for accessing the instance.
 - HTTP (port 80) for the Nagios web interface.
- Launch the instance and connect via SSH using your preferred method (EC2 Instance Connect, SSH from your terminal, etc.).



aws

Services

Q Search

[Alt+S]

N. Virginia

Rohan_Lalchandani

▼ Application and OS Images (Amazon Machine Image) Info

An AMI is a template that contains the software configuration (operating system, application server, and applications) required to launch your instance. Search or Browse for AMIs if you don't see what you are looking for below

Q Search our full catalog including 1000s of application and OS images

RecentsQuick Start

Amazon Linux

macOS

Ubuntu

Windows

Red Hat

SUSE Li

Browse more AMIs

Amazon Machine Image (AMI)

Amazon Linux 2023 AMI
ami-06b21c9aeff8cd686 (64-bit (x86), uefi-preferred) / ami-02801556a781a4499 (64-bit (Arm), uefi)
Virtualization: hvm ENA enabled: true Root device type: ebs

Free tier eligible

▼ Summary

Number of instances Info

1

Software Image (AMI)

Amazon Linux 2023 AMI 2023.6.2...read more
ami-06b21c9aeff8cd686

Virtual server type (instance type)

t2.micro

Firewall (security group)

New security group

Storage (volumes)

1 volume(s) - 8 GiB

CancelLaunch instance

Preview code

aws

Services

Q Search

[Alt+S]

N. Virginia

Rohan_Lalchandani

Architecture64-bit (x86)

Boot modeuefi-preferred

AMI IDami-06b21c9aeff8cd686

Usernameec2-user

Verified provider

▼ Instance type Info | Get advice

Instance type

t2.micro

Family: t2 1 vCPU 1 GiB Memory Current generation: true

On-Demand Windows base pricing: 0.0162 USD per Hour

On-Demand Ubuntu Pro base pricing: 0.0134 USD per Hour

On-Demand SUSE base pricing: 0.0116 USD per Hour

On-Demand RHEL base pricing: 0.026 USD per Hour

On-Demand Linux base pricing: 0.0116 USD per Hour

Free tier eligible

All generations

Compare instance types

Additional costs apply for AMIs with pre-installed software

▼ Key pair (login) Info

▼ Summary

Number of instances Info

1

Software Image (AMI)

Amazon Linux 2023 AMI 2023.6.2...read more
ami-06b21c9aeff8cd686

Virtual server type (instance type)

t2.micro

Firewall (security group)

New security group

Storage (volumes)

1 volume(s) - 8 GiB

CancelLaunch instance

Preview code

aws

Services

Q Search

[Alt+S]

N. Virginia

Rohan_Lalchandani

▼ Key pair (login) Info

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - required

Amazon_Linux

Create new key pair

▼ Network settings Info

VPC - required Info

vpc-0c35a6d76aeb8bb99

(default)

Subnet Info

No preference

Create new subnet

Auto-assign public IP Info

▼ Summary

Number of instances Info

1

Software Image (AMI)

Amazon Linux 2023 AMI 2023.6.2...read more
ami-06b21c9aeff8cd686

Virtual server type (instance type)

t2.micro

Firewall (security group)

New security group

Storage (volumes)

1 volume(s) - 8 GiB

CancelLaunch instance

Preview code

aws

Services

Search

[Alt+S]

Instance

Create security group

Select existing security group

Security group name - required

linux_Rohan

This security group will be added to all network interfaces. The name can't be edited after the security group is created. Max length is 255 characters. Valid characters: a-z, A-Z, 0-9, spaces, and _-./!@#%&'()*+<=>:[]{}*~

Description - required

linux_Rohan

Inbound Security Group Rules

Security group rule 1 (TCP, 22, 0.0.0.0/0)

Remove

Type

ssh

Protocol

TCP

Port range

22

Source type

Anywhere

Source

0.0.0.0/0

Description - optional

e.g. SSH for admin desktop

Summary

Number of instances

1

Software Image (AMI)

Amazon Linux 2023 AMI 2023.6.2...read more

ami-06b21ccea8f8cd686

Virtual server type (instance type)

t2.micro

Firewall (security group)

New security group

Storage (volumes)

1 volume(s) - 8 GiB

Cancel

Launch instance

Preview code

aws

Services

Search

[Alt+S]

Anywhere

Add CIDR, prefix list or security

e.g. SSH for admin desktop

0.0.0.0/0

Security group rule 2 (TCP, 80, 0.0.0.0/0)

Remove

Type

HTTP

Protocol

TCP

Port range

80

Source type

Anywhere

Source

0.0.0.0/0

Description - optional

e.g. SSH for admin desktop

Rules with source of 0.0.0.0/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

Add security group rule

Summary

Number of instances

1

Software Image (AMI)

Amazon Linux 2023 AMI 2023.6.2...read more

ami-06b21ccea8f8cd686

Virtual server type (instance type)

t2.micro

Firewall (security group)

New security group

Storage (volumes)

1 volume(s) - 8 GiB

Cancel

Launch instance

Preview code

aws

Services

Search

[Alt+S]

EC2 Dashboard

EC2 Global View

Events

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity

Reservations

Images

AMIs

AMI Catalog

Instances (1) Info

Last updated less than a minute ago

Connect

Instance state

Actions

Launch instances

Find Instance by attribute or tag (case-sensitive)

All states

	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability
☐	linux_Rohan	i-08c9b39161d8ebd41	Running	t2.micro	Initializing	View alarms	us-east-1d

Select an instance

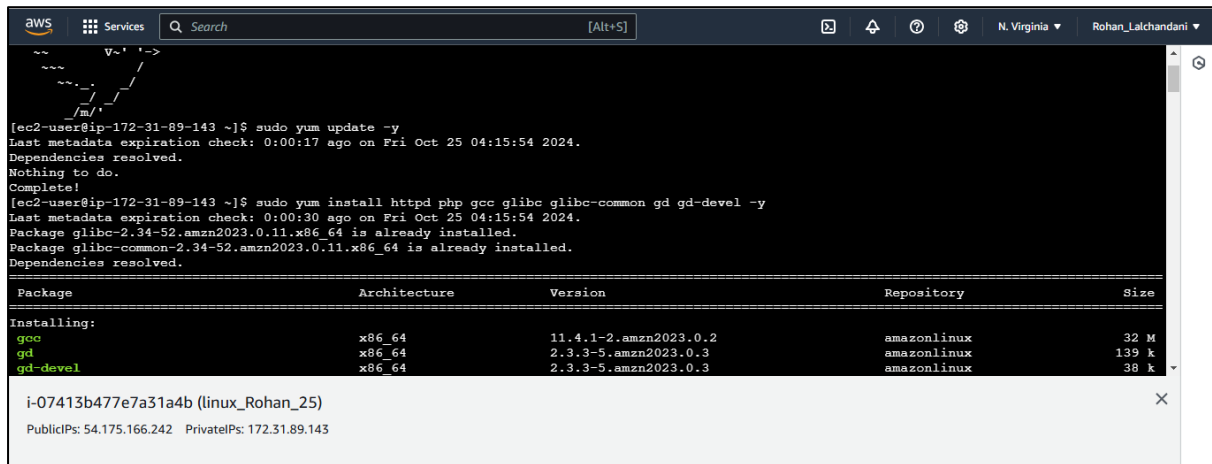
Step 2: Install Nagios on EC2

1. Update System Packages:

```
sudo yum update -y
```

Install Prerequisites: Install development tools like Apache, PHP, and required libraries:

```
sudo yum install httpd php gcc glibc glibc-common gd gd-devel -y
```



```
aws
Services
Search [Alt+S]
N. Virginia Rohan_Lalchandani

[ec2-user@ip-172-31-89-143 ~]$ sudo yum update -y
Last metadata expiration check: 0:00:17 ago on Fri Oct 25 04:15:54 2024.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31-89-143 ~]$ sudo yum install httpd php gcc glibc glibc-common gd gd-devel -y
Last metadata expiration check: 0:00:30 ago on Fri Oct 25 04:15:54 2024.
Package glibc-2.34-52.amzn2023.0.11.x86_64 is already installed.
Package glibc-common-2.34-52.amzn2023.0.11.x86_64 is already installed.
Dependencies resolved.

Package Architecture Version Repository Size
Installing:
gcc x86_64 11.4.1-2.amzn2023.0.2 amazonlinux 32 M
gd x86_64 2.3.3-5.amzn2023.0.3 amazonlinux 139 k
gd-devel x86_64 2.3.3-5.amzn2023.0.3 amazonlinux 38 k

i-07413b477e7a31a4b (linux_Rohan_25)
PublicIPs: 54.175.166.242 PrivateIPs: 172.31.89.143
```

Create Nagios User and Group:

```
sudo useradd nagios
sudo groupadd nagcmd
sudo usermod -aG nagcmd nagios
sudo usermod -aG nagcmd apache
```



```
Complete!
[ec2-user@ip-172-31-89-143 ~]$ sudo useradd nagios
sudo groupadd nagcmd
sudo usermod -aG nagcmd nagios
sudo usermod -aG nagcmd apache
[ec2-user@ip-172-31-89-143 ~]$

i-07413b477e7a31a4b (linux_Rohan_25)
PublicIPs: 54.175.166.242 PrivateIPs: 172.31.89.143
```

Download and Install Nagios Core:

```
cd /tmp
wget https://assets.nagios.com/downloads/nagioscore/releases/nagios-4.4.6.tar.gz
tar xzf nagios-4.4.6.tar.gz
cd nagios-4.4.6
sudo ./configure --with-command-group=nagcmd
sudo make all
sudo make install
sudo make install-init
sudo make install-config
sudo make install-commandmode
sudo make install-webconf
```

```
aws Services Search [Alt+S]
[ec2-user@ip-172-31-89-143 ~]$ sudo useradd nagios
sudo groupadd nagcmd
sudo usermod -aG nagcmd nagios
sudo usermod -aG nagcmd apache
[ec2-user@ip-172-31-89-143 ~]$ cd /tmp
wget https://assets.nagios.com/downloads/nagioscore/releases/nagios-4.4.6.tar.gz
tar xzf nagios-4.4.6.tar.gz
cd nagios-4.4.6
sudo ./configure --with-command-group=nagcmd
sudo make all
sudo make install
sudo make install-init
sudo make install-config
sudo make install-commandmode
sudo make install-webconf
--2024-10-25 04:18:21-- https://assets.nagios.com/downloads/nagioscore/releases/nagios-4.4.6.tar.gz
Resolving assets.nagios.com (assets.nagios.com)... 45.79.49.120, 2600:3c00::f03c:92ff:fef7:45ce
Connecting to assets.nagios.com (assets.nagios.com)|45.79.49.120|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 11333414 (11M) [application/x-gzip]
Saving to: 'nagios-4.4.6.tar.gz'
```

i-07413b477e7a31a4b (linux_Rohan_25)
PublicIPs: 54.175.166.242 PrivateIPs: 172.31.89.143

Install and Configure Nagios Plugins:

```
cd /tmp
wget https://nagios-plugins.org/download/nagios-plugins-2.3.3.tar.gz
tar xzf nagios-plugins-2.3.3.tar.gz
cd nagios-plugins-2.3.3
sudo ./configure --with-nagios-user=nagios --with-nagios-group=nagios
sudo make
sudo make install
```

```
[ec2-user@ip-172-31-89-143 nagios-4.4.6]$ cd /tmp
wget https://nagios-plugins.org/download/nagios-plugins-2.3.3.tar.gz
tar xzf nagios-plugins-2.3.3.tar.gz
cd nagios-plugins-2.3.3
sudo ./configure --with-nagios-user=nagios --with-nagios-group=nagios
sudo make
sudo make install
--2024-10-25 04:19:30-- https://nagios-plugins.org/download/nagios-plugins-2.3.3.tar.gz
Resolving nagios-plugins.org (nagios-plugins.org)... 45.56.123.251
Connecting to nagios-plugins.org (nagios-plugins.org)|45.56.123.251|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 2782610 (2.7M) [application/x-gzip]
```

i-07413b477e7a31a4b (linux_Rohan_25)
PublicIPs: 54.175.166.242 PrivateIPs: 172.31.89.143

Configure Nagios Web Interface:

Set up an admin user for Nagios:

```
sudo htpasswd -c /usr/local/nagios/etc/htpasswd.users nagiosadmin
```

(You'll be prompted to create a password for the user nagiosadmin).

```
[ec2-user@ip-172-31-89-143 nagios-plugins-2.3.3]$ sudo htpasswd -c /usr/local/nagios/etc/htpasswd.users nagiosadmin
New password:
Re-type new password:
Adding password for user nagiosadmin
[ec2-user@ip-172-31-89-143 nagios-plugins-2.3.3]$
```

i-07413b477e7a31a4b (linux_Rohan_25)
PublicIPs: 54.175.166.242 PrivateIPs: 172.31.89.143

Restart Apache:

```
sudo systemctl restart httpd
```

Start and Enable Nagios:

```
sudo systemctl start nagios
```

```
sudo systemctl enable nagios
```

```
[ec2-user@ip-172-31-89-143 nagios-plugins-2.3.3]$ sudo systemctl restart httpd
[ec2-user@ip-172-31-89-143 nagios-plugins-2.3.3]$ sudo systemctl start nagios
sudo systemctl enable nagios
Created symlink /etc/systemd/system/multi-user.target.wants/nagios.service → /usr/lib/systemd/system/nagios.service.
[ec2-user@ip-172-31-89-143 nagios-plugins-2.3.3]$
```

i-07413b477e7a31a4b (linux_Rohan_25)

PublicIPs: 54.175.166.242 PrivateIPs: 172.31.89.143

Install Nagios Plugin for CloudWatch Logs (AWS CloudWatch Integration):

You need a Nagios plugin that can query AWS CloudWatch Logs. A commonly used solution is the **CloudWatch plugin** for Nagios.

- Install dependencies (for Nagios to talk to AWS):

```
sudo apt-get install python3-pip
```

```
pip3 install boto3
```

```
[ec2-user@ip-172-31-89-143 nagios-plugins-2.3.3]$ sudo yum install python3 -y
Last metadata expiration check: 0:10:55 ago on Fri Oct 25 04:15:54 2024.
Package python3-3.9.16-1.amzn2023.0.9.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
```

```
[ec2-user@ip-172-31-89-143 nagios-plugins-2.3.3]$ sudo yum install python3-pip -y
Last metadata expiration check: 0:12:47 ago on Fri Oct 25 04:15:54 2024.
Dependencies resolved.
```

Package	Architecture	Version	Repository	Size
Installing:				
python3-pip	noarch	21.3.1-2.amzn2023.0.8	amazonlinux	1.8 M
Installing weak dependencies:				
libxcrypt-compat	x86_64	4.4.33-7.amzn2023	amazonlinux	92 k

```
Transaction Summary
Install 2 Packages
Total download size: 1.9 M
Installed size: 11 M
Downloading Packages:
(1/2): libxcrypt-compat-4.4.33-7.amzn2023.x86_64.rpm 1.5 MB/s | 92 kB 00:00
(2/2): python3-pip-21.3.1-2.amzn2023.0.8.noarch.rpm 19 MB/s | 1.8 MB 00:00
```

i-07413b477e7a31a4b (linux_Rohan_25)

PublicIPs: 54.175.166.242 PrivateIPs: 172.31.89.143

Use AWS CLI and Custom Nagios Plugin

Install AWS CLI (if not already installed):

```
[ec2-user@ip-172-31-89-143 libexec]$ sudo yum install awscli -y
Last metadata expiration check: 0:22:21 ago on Fri Oct 25 04:15:54 2024.
Package awscli-2.2.15.30-1.amzn2023.0.1.noarch is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31-89-143 libexec]$
```

i-07413b477e7a31a4b (linux_Rohan_25)

PublicIPs: 54.175.166.242 PrivateIPs: 172.31.89.143

Test AWS CLI configuration: If you haven't already configured your AWS CLI, run:

Provide your AWS Access Key, Secret Access Key, default region, and output format (e.g., json).

```
[ec2-user@ip-172-31-89-143 libexec]$ aws configure
AWS Access Key ID [None]: AKIA4RCAOUQS7WRUQ5HK
AWS Secret Access Key [None]: s0qlMP00HDL8Nc4Kt5bwIviJ07Ygji/XVdHlM5na
Default region name [None]: us-east-1
Default output format [None]: json
[ec2-user@ip-172-31-89-143 libexec]$
```

i-07413b477e7a31a4b (linux_Rohan_25)

PublicIPs: 54.175.166.242 PrivateIPs: 172.31.89.143

Create a Custom Script to Check CloudWatch Logs: You can create a custom shell script to fetch CloudWatch log data from your Lambda function using the AWS CLI. Here's an example script:

```
sudo nano /usr/local/nagios/libexec/check_lambda_logs.sh
```

Add the following content:

```
#!/bin/bash
```

```
# Custom Nagios plugin to monitor AWS CloudWatch Logs for Lambda function
```

```
LOG_GROUP_NAME="/aws/lambda/lambda_Rohan_25"
```

```
FILTER_PATTERN="ERROR"
```

```
AWS_REGION="your-region" # Example: us-east-1
```

```
# Fetch CloudWatch logs using AWS CLI
```

```
LOGS=$(aws logs filter-log-events --log-group-name "$LOG_GROUP_NAME" --filter-pattern  
"$FILTER_PATTERN" --region "$AWS_REGION" --limit 1)
```

```
if [[ -n "$LOGS" ]]; then
```

```
    echo "CRITICAL: Errors found in Lambda logs!"
```

```
    exit 2
```

```
else
```

```
    echo "OK: No errors in Lambda logs."
```

```
    exit 0
```

```
fi
```

Make the script executable:

```
sudo chmod +x /usr/local/nagios/libexec/check_lambda_logs.sh
```

Test the script manually:

```
/usr/local/nagios/libexec/check_lambda_logs.sh
```

Add the script to Nagios: Define the command in Nagios:

Open your commands.cfg file:

```
sudo nano /usr/local/nagios/etc/objects/commands.cfg
```

```
[ec2-user@ip-172-31-89-143 libexec]$ sudo nano /usr/local/nagios/libexec/check_lambda_logs.sh
[ec2-user@ip-172-31-89-143 libexec]$ sudo chmod +x /usr/local/nagios/libexec/check_lambda_logs.sh
[ec2-user@ip-172-31-89-143 libexec]$ /usr/local/nagios/libexec/check_lambda_logs.sh
```

```
[ec2-user@ip-172-31-89-143 libexec]$ /usr/local/nagios/libexec/check_lambda_logs.sh
OK: No errors in Lambda logs.
[ec2-user@ip-172-31-89-143 libexec]$
```

i-07413b477e7a31a4b (linux_Rohan_25)

PublicIPs: 54.175.166.242 PrivateIPs: 172.31.89.143

Add this command definition:

```
define command{
    command_name    check_lambda_logs
    command_line    /usr/local/nagios/libexec/check_lambda_logs.sh
}
```

```
[ec2-user@ip-172-31-89-143 libexec]$ sudo nano /usr/local/nagios/etc/objects/commands.cfg
[ec2-user@ip-172-31-89-143 libexec]$
```

i-07413b477e7a31a4b (linux_Rohan_25)

PublicIPs: 54.175.166.242 PrivateIPs: 172.31.89.143

```
[ec2-user@ip-172-31-89-143 libexec]$ sudo nano /usr/local/nagios/etc/objects/localhost.cfg
[ec2-user@ip-172-31-89-143 libexec]$
```

i-07413b477e7a31a4b (linux_Rohan_25)

PublicIPs: 54.175.166.242 PrivateIPs: 172.31.89.143

Add a service in Nagios to monitor the logs: Add this service definition in your Nagios configuration (e.g., localhost.cfg):

```
define service{
    use                generic-service
    host_name          localhost
    service_description Monitor Lambda Logs (lambda_Rohan_25)
    check_command       check_lambda_logs
}
```

```
aws Services [Alt+S] N. Virginia Rohan_Lalchandani
GNU nano 5.8 /usr/local/nagios/etc/objects/localhost.cfg Modified
define service{
    use                generic-service      ; Inherits properties from generic template
    host_name          localhost            ; The host where the script runs
    service_description Monitor Lambda Logs (lambda_Rohan_25)
    check_command       check_lambda_logs   ; Command name from commands.cfg
    check_interval      5                  ; Check every 5 minutes
    retry_interval      1                  ; Retry every 1 minute if the check fails
    max_check_attempts  3                  ; Allow 3 retries before marking it critical
    notification_interval 30                ; Notify every 30 minutes
    notification_options w,c,r              ; Notify on warning, critical, and recovery states
}
#####
# LOCALHOST.CFG - SAMPLE OBJECT CONFIG FILE FOR MONITORING THIS MACHINE
#
# NOTE: This config file is intended to serve as an "extremely" simple
#       example of how you can create configuration entries to monitor
#       the local (Linux) machine.
#
^G Help      ^O Write Out ^W Where Is  ^R Cut        ^E Execute   ^G Location  ^U Undo      ^M Set Mark  ^J To Bracket
^X Exit      ^R Read File ^N Replace   ^P Paste     ^V Justify   ^_ Go To Line  ^-E Redo    ^-G Copy     ^_ Where Was

i-07413b477e7a31a4b (linux_Rohan_25)
```

PublicIPs: 54.175.166.242 PrivateIPs: 172.31.89.143

```
Checked 1 hosts
Checked 0 service dependencies
Checked 0 host dependencies
Checked 5 timeperiods
Checking global event handlers...
Checking obsessive compulsive processor commands...
Checking misc settings...

Total Warnings: 0
Total Errors: 0

Things look okay - No serious problems were detected during the pre-flight check
[ec2-user@ip-172-31-89-143 ~]$
```

i-07413b477e7a31a4b (linux_Rohan_25)

PublicIPs: 54.175.166.242 PrivateIPs: 172.31.89.143

Restart Nagios: Verify the configuration and restart Nagios:

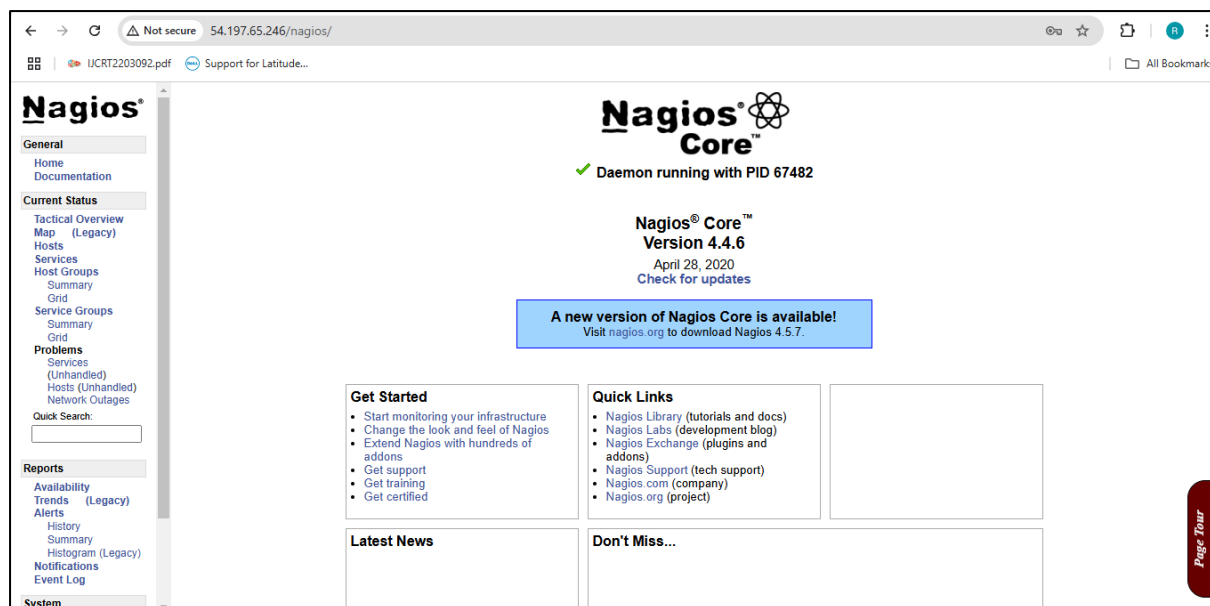
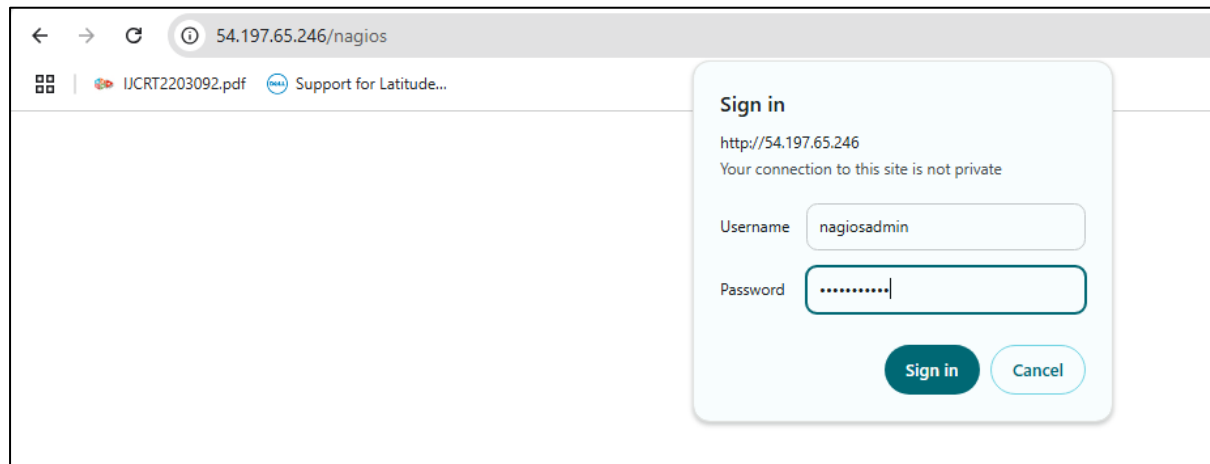
```
sudo /usr/local/nagios/bin/nagios -v /usr/local/nagios/etc/nagios.cfg
sudo systemctl restart nagios
```

Run this command to see logs in nagios

```
[ec2-user@ip-172-31-95-127 libexec]$ /usr/local/nagios/libexec/check_lambda_logs.py
Checking log stream: 2024/10/25/[$LATEST]aa5d5871ed4d487ebe0432ab9854d1c2
OK - Lambda logs found in CloudWatch, recent upload detected.
[ec2-user@ip-172-31-95-127 libexec]$
```

i-08c9b39161d8ebd41 (linux_Rohan)

PublicIPs: 54.197.65.246 PrivateIPs: 172.31.95.127

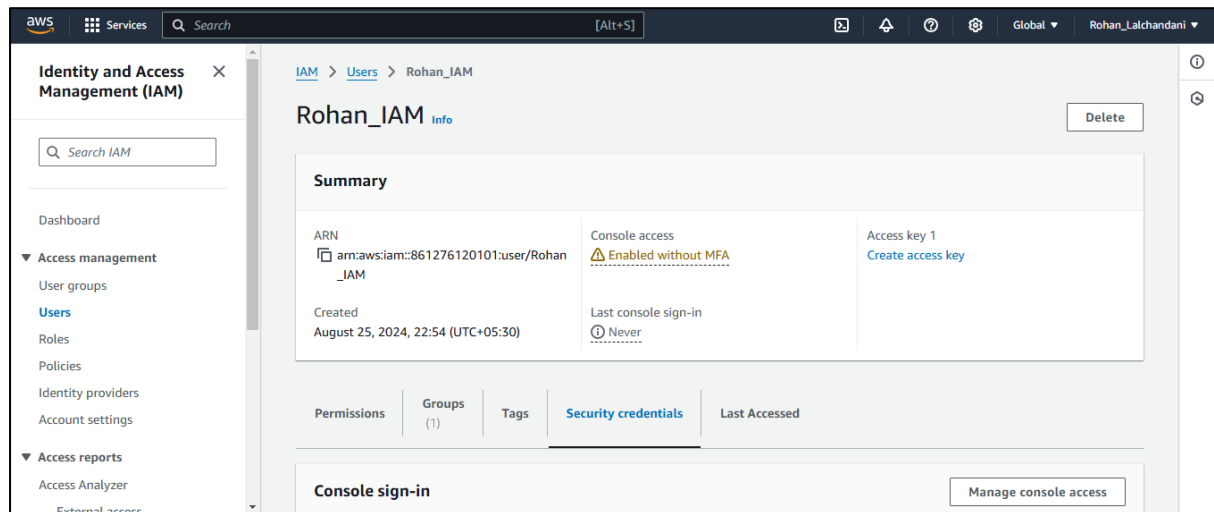


If Facing any issues in IAM policies, access denied error follow this

Step 1: Create or Find AWS Access Key ID and Secret Access Key

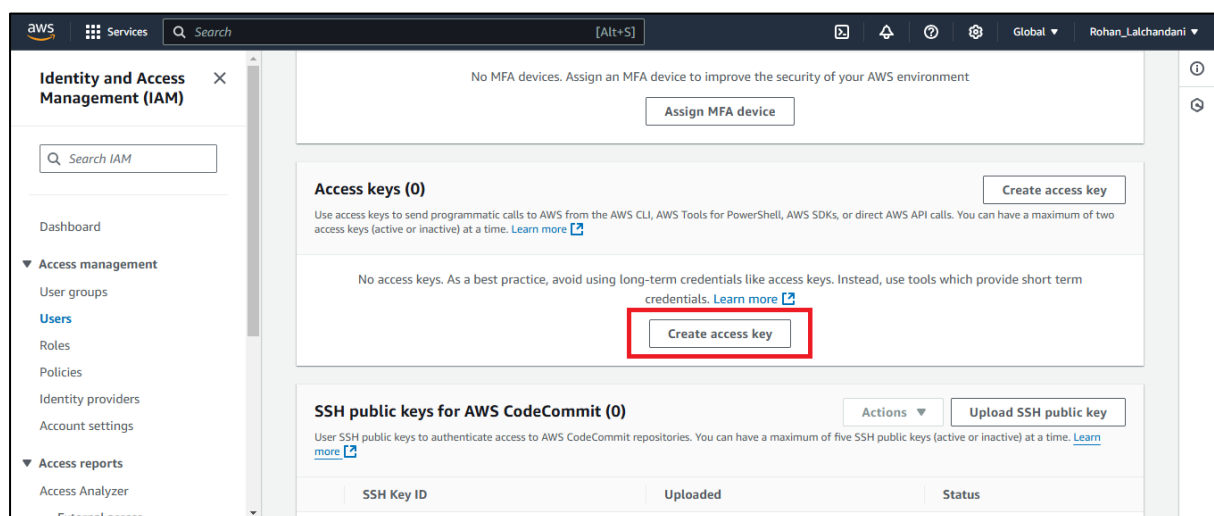
1. Login to AWS Management Console:

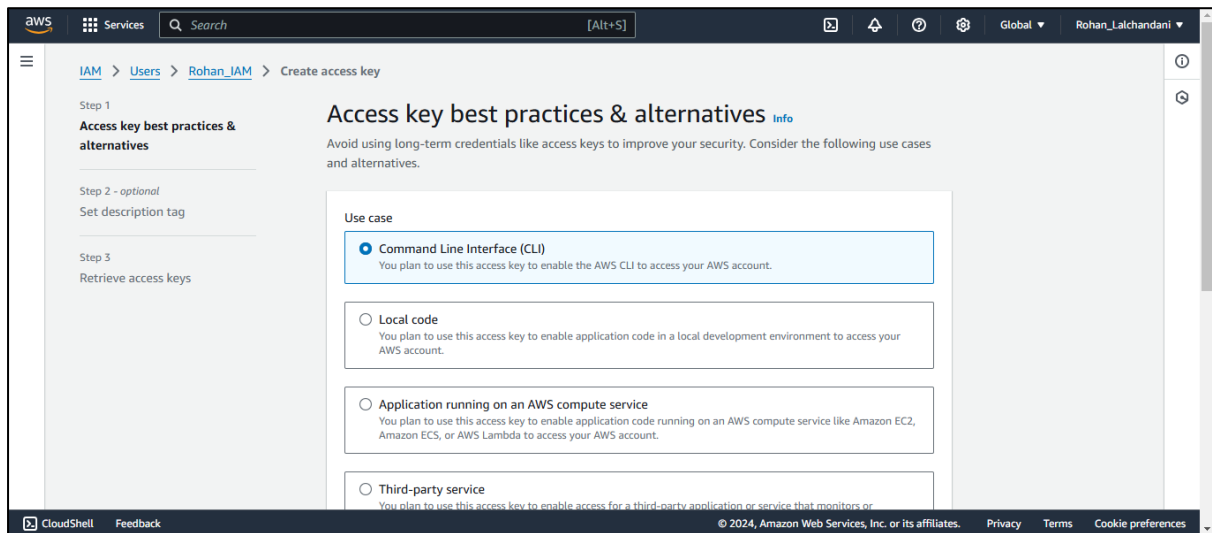
- Go to [AWS Management Console](#).
- Login with your credentials.



1. Create a New Access Key (if you don't have one):

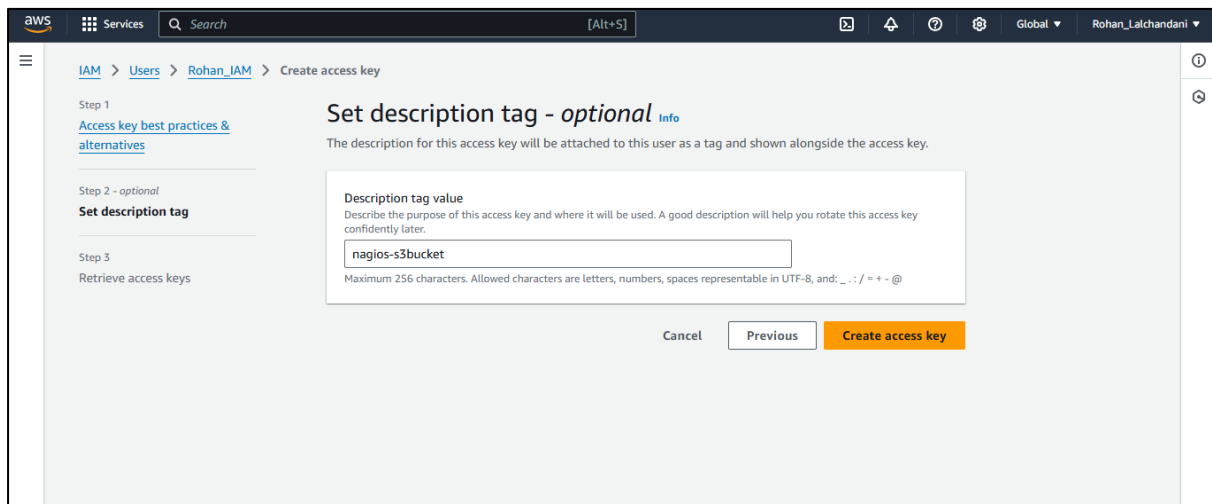
- In the left panel, click on **Users**.
- Select your username from the list.
- Under the **Security Credentials** tab, scroll down to **Access Keys**.
- Click **Create access key** if you don't already have one.





Copy the Access Key ID and Secret Access Key:

- After creating the access key, AWS will display the **Access Key ID** and **Secret Access Key**. **Be sure to copy and store the Secret Access Key**, as AWS will not show it again after the initial creation.



Implementation Details

AWS Services Utilized:

- **AWS Lambda:**
 - AWS Lambda allows developers to run code in response to events without provisioning or managing servers. In this case study, a Lambda function is set up to handle image processing tasks triggered by uploads to an S3 bucket.
- **Amazon S3:**
 - Amazon Simple Storage Service (S3) provides durable and highly available object storage. It is used to store the uploaded images securely and facilitates easy access for processing by Lambda.
- **Nagios:**
 - Nagios is a powerful monitoring system that enables organizations to identify and resolve IT infrastructure problems before they affect critical business processes. In this case, Nagios monitors the health of the AWS resources and triggers alerts based on predefined thresholds.

Workflow Steps:

1. **Image Upload:**
 - Users upload images to the designated S3 bucket, triggering an S3 event.
2. **Triggering Lambda Function:**
 - The upload event triggers the AWS Lambda function, which is configured to respond to specific events in the S3 bucket.
3. **Image Processing:**
 - The Lambda function processes the images as needed (e.g., resizing, format conversion, or generating thumbnails).
4. **Monitoring:**
 - Nagios monitors the performance and availability of the application, providing alerts for any issues, such as failed Lambda executions or S3 bucket access problems.